PROJECT SYNOPSIS & BUILD SPECIFICATION

RevOpsIQ

B2B SaaS Strategy & Operations Intelligence Platform

Purpose
A functional internal operating platform connecting market intelligence, account prioritisation, sales pipeline operations, customer onboarding and adoption, business performance, business reviews and prioritised execution.

Audience
A developer or AI coding agent responsible for planning, implementing, testing, documenting and delivering the application.

Core instruction: Build a working business application, not a static dashboard, slide deck or collection of disconnected mock screens. Every important workflow must be backed by persistent data, defined business logic and testable behaviour.

Document map

Section	Contents
1	Executive summary and product definition
2	Problem, goals and non-goals
3	Users, roles and permissions
4	Six functional modules
5	End-to-end workflows
6	Data model and entities
7	Metrics, formulas and decision logic
8	AI / recommendation layer and guardrails
9	Technical architecture and repository structure
10	UI / UX requirements
11	Demo data and import strategy
12	Security and reliability
13	Testing and acceptance criteria
14	Implementation phases
15	AI-agent operating instructions
16	Deliverables, demo scenarios and definition of done

Assumptions must be explicit and configurable. Synthetic data, market estimates and illustrative calculations must never be presented as verified facts.

1. Executive summary and product definition

Product name: RevOpsIQ (working name). **Category:** B2B SaaS revenue operations, strategy and business operations intelligence platform.

RevOpsIQ is a web application for a fictional or configurable B2B SaaS company. It centralises market and competitor intelligence, target-account research, opportunity pipeline tracking, customer onboarding and product adoption, revenue and unit-economics metrics, business reviews, and operational action tracking.

Continuous operating loop: Understand the market → prioritise accounts → manage opportunities → onboard customers → monitor adoption and revenue → review performance → assign actions → measure follow-through.

Modules must share data. A market segment links to accounts; accounts link to opportunities; Closed Won opportunities convert to customers; customers have onboarding and adoption records; customer and deal data feed business metrics; insights create recommendations and assigned actions.

This is a portfolio-grade prototype, not a production CRM replacement. It must implement real CRUD operations, persistence, calculations, filtering, workflows, validation and export. Static mock data is permitted only as clearly labelled seeded demo data.

Primary product outcome

Help a business operations team decide where to focus, understand what is happening and why, and turn insights into assigned, trackable actions.

2. Problem, goals and non-goals

B2B SaaS teams often manage market research, account lists, sales opportunities, onboarding milestones, product usage, financial performance and weekly action items in separate tools. Fragmentation makes it difficult to maintain a consistent business view, identify bottlenecks, compare segments, surface customer risk and ensure decisions lead to execution.

Goals

- Centralise core business records in a relational data model.
- Support market and competitor research with source notes, assumptions and segment scoring.
- Support account qualification and opportunity tracking with transparent scoring and stage history.

- Track onboarding, adoption signals, customer health, risks and follow-up actions.
- Calculate SaaS business metrics using documented formulas and traceable inputs.
- Generate business review views comparing targets with actuals and highlighting exceptions.
- Convert insights into owned actions with priority, due date, status and completion evidence.
- Provide audit history for important changes and recommendation decisions.
- Support seeded demo data and CSV import.

Non-goals

- Do not replace Salesforce, HubSpot, Gainsight, an ERP or accounting system.
- Do not connect to real customer accounts, send emails, issue invoices or initiate payments.
- Do not scrape websites in violation of terms or bypass authentication.
- Do not label synthetic data or illustrative assumptions as verified facts.
- Do not claim scores are validated predictions without evaluation.
- Do not autonomously reject leads, terminate customers or change commercial terms.

3. Users, roles and permissions

Role	Responsibilities	Permissions
Admin	Configure workspace, users, scoring weights, targets and reference lists.	Full access; manage settings and users.
Strategy / Business Ops	Research markets, review performance, coordinate projects and manage actions.	Read all modules; create/edit research, reviews and actions.
Sales	Manage accounts, contacts, opportunities, stages and next steps.	Edit assigned accounts and opportunities; view market data.
Customer Success	Manage onboarding, adoption, health and risks.	Edit customer records, milestones, health events and follow-ups.
Executive / Viewer	Review performance, risks, opportunities and decisions.	Read-only access.

Backend authorization is mandatory. Hiding buttons in the frontend is not sufficient. Preserve historical records through archive/soft-delete where appropriate. Log record creator, updater and timestamps. Seeded demo credentials must be documented and must not be real credentials.

4. Six functional modules

Module 1 — Market Intelligence

Purpose: Understand which industries, segments and competitor positions may represent attractive opportunities. Screens: Market Overview, Segments, Competitors, Research Sources, Opportunity Scoring and Segment Detail.

- Create, edit, archive, search and filter market segments. Fields: name, industry, geography, company-size band, use case, estimated target-organisation count, urgency/problem indicators, buying complexity, notes and status.
- Store TAM, SAM and SOM with currency, period, method, assumptions, source, confidence and review date. Estimates must never appear as verified facts without methodology and source.
- Create competitor profiles: company, website, target segments, product category, stated capabilities, pricing notes, differentiators, strengths, weaknesses, source URL, observation date and confidence.
- Compare competitors in a matrix. Distinguish sourced facts from analyst observations.
- Score segments with configurable criteria: market attractiveness, urgency, product fit, accessibility and competitive intensity. Show score breakdown and rationale.
- Link target accounts to one or more segments.

Market sizing: Support top-down or bottom-up methods selected by the user. Example bottom-up calculation: number of target organisations × estimated annual contract value. Display assumptions and method next to the result.

Acceptance: A user can create a segment, attach evidence, compare competitors, calculate a transparent score and link target accounts.

Module 2 — Account Intelligence & Sales Pipeline

Purpose: Maintain target account records and track opportunities from research through close. Screens: Account Directory, Account Detail, Account Scorecard, Opportunity Pipeline, Deal Detail, Pipeline Forecast and Stalled Deals.

- Account fields: organisation, domain, industry, geography, employee/revenue band (optional), segment, use case, pain point, fit notes, source, owner, status, priority and last-researched date.
- Contact fields: name, role, department, optional business email, account, source and notes. Do not require personal contact data for seeded demo records.
- Configurable account-fit score using segment fit, use-case fit, size fit, geography fit, urgency and data completeness. Show component breakdown; label as rule-based, not ML prediction.
- Opportunity fields: name, account, owner, stage, estimated deal value, currency, probability, expected close date, source, next step, next-step due date, last activity, loss reason and notes.
- Default stages: Target → Qualified → Discovery → Evaluation / Demo → Proposal → Negotiation → Closed Won / Closed Lost. Admin can configure stages and probabilities.
- Provide Kanban and sortable table views; retain stage history with timestamp and actor.
- Weighted pipeline = open deal value × probability. Label as estimate, not booked revenue.
- Flag stale opportunities, missing next steps, overdue next steps and past expected close dates.
- Filter by owner, segment, stage, geography, deal size and close period.
- Closed Won can create/link a customer without duplicating the account. Closed Lost records loss reason and optional competitor.

Acceptance: Create and score an account, create an opportunity, move it through stages, record next steps, view weighted pipeline and identify stalled deals.

Module 3 — Customer Success, Onboarding & Adoption

Purpose: Track the transition from Closed Won to successful onboarding and ongoing adoption. Screens: Customer Directory, Customer Detail, Onboarding Board, Adoption Overview and Health & Risk Queue.

- Customer fields: linked account, source opportunity, start date, plan/tier, contract value, renewal date, owner and lifecycle status.
- Configurable onboarding milestones: Kickoff Scheduled, Requirements / Configuration, Integration, Testing, Training, Go-Live and Adoption Review.
- Each milestone has owner, due date, status, completion date, blocker and notes.
- Track adoption observations such as active users, feature usage, usage frequency, successful workflows or training completion. Data may be manually entered or CSV-imported; do not imply live telemetry integration.
- Calculate transparent health score from configurable components: adoption, onboarding progress, support issues, engagement and renewal proximity. Show component contributions and missing-data warnings.
- Flag falling usage, overdue onboarding and upcoming renewal for review.
- Track customer risks, mitigation plan, next contact date and owner. Create follow-up actions linked to customer.

Acceptance: Convert a won deal into a customer, create onboarding milestones, record adoption, inspect health/risk explanation and assign follow-up actions.

Module 4 — Business & Financial Performance

Purpose: Translate pipeline, customer and financial records into a consistent business-performance view. Screens: Executive Overview, Revenue, Pipeline Performance, Segment Performance, Customer Economics and Targets vs Actuals.

Metric	Calculation / definition	Qualification
Bookings / Won Deal Value	Sum Closed Won opportunity values for selected period.	Use close date and consistent currency.
Pipeline Value	Sum of open opportunity deal values.	Not revenue.
Weighted Pipeline	Sum of open deal value × probability.	Estimate, not guaranteed.
Win Rate	Closed Won ÷ (Won + Lost) × 100.	Define period/cohort.
Stage Conversion	Count entering next stage ÷ count entering prior stage × 100.	Use consistent cohort logic.
Sales Cycle	Average days from defined qualified date to Closed Won.	State start/end events.
MRR / ARR	Recurring contract value normalised monthly / annually.	Only when recurring terms exist.
CAC	Acquisition sales and marketing cost ÷ new customers.	If costs absent, show Not Available.

Metric	Calculation / definition	Qualification
LTV	Estimated gross profit per customer over expected lifetime.	Label estimated; expose assumptions.
Gross Margin	(Revenue – cost of service) ÷ revenue × 100.	Never infer missing costs.
Customer Churn	Customers lost ÷ customers at period start × 100.	Define logo vs revenue churn.
Net Revenue Retention	(Opening recurring revenue + expansion – contraction – churn) ÷ opening recurring revenue × 100.	Requires complete revenue movement data.

- Support monthly and quarterly targets by metric, segment, owner and period.
- Show actual, target, absolute variance and percentage variance.
- Allow drill-down from KPI to source records.
- Filter by date, segment, geography, owner and cohort.
- Never aggregate currencies without explicit conversion rates and conversion-date basis.
- Handle missing values and zero denominators safely.
- Provide metric glossary and CSV export; provide PDF/print-friendly business summary.

Acceptance: Inspect KPIs, compare actuals to targets, filter by segment/period and trace metrics to source records.

Module 5 — Business Reviews & Operations

Purpose: Turn performance into repeatable reviews, decisions, accountable actions and follow-through. Screens: Weekly Review, Monthly Business Review, Review Detail, Action Tracker, Project Tracker and Blockers & Risks.

- Create reviews with period, title, date, participants, owner and status.
- Attach KPI snapshots, segment insights, pipeline updates, customer risks and project updates.
- Record discussion points, decisions, assumptions, unresolved questions and dependencies.
- Actions contain title, description, owner, priority, due date, status, linked entity, created date and completion notes.
- Action statuses: Not Started, In Progress, Blocked, Completed, Cancelled. Priorities: Critical, High, Medium, Low.
- Show overdue, blocked, due-this-week and completed actions.
- Project/workstream records contain objective, owner, milestones, dates, progress, risks and status.
- Review states: Draft, In Review, Finalised. Finalised reviews preserve KPI snapshots so history does not silently change.

Acceptance: Create a weekly review, capture decisions, assign actions, monitor overdue work and inspect completion history.

Module 6 — Strategy & Decision Engine

Purpose: Bring signals from the other modules together and propose explainable, prioritised next actions. Start with deterministic, testable rules. Every recommendation must show trigger, evidence, rationale, severity, suggested owner role, suggested due date and Accept / Dismiss / Snooze / Convert to Action controls.

Condition	Suggested action	Evidence shown
High-fit segment, few target accounts	Research/add target accounts.	Segment score, account count, source coverage.
High-fit account without opportunity	Review account and qualify/create opportunity if supported.	Fit score and components.
Opportunity inactive beyond threshold	Review status and schedule next step.	Last activity, stage, owner, close date.
Overdue opportunity next step	Assign follow-up and revise close plan.	Due date, owner, stage.
Pipeline concentrated in one segment	Review concentration and diversify prospecting.	Pipeline by segment and share.
Onboarding milestone overdue	Escalate blocker and update plan.	Milestone, due date, blocker, owner.
Adoption below threshold	Initiate customer health review.	Usage trend and health components.
Renewal near with low health	Assign retention review.	Renewal date, health score, risk.
Actual below target	Investigate variance by segment/owner/stage.	Target, actual, variance.
Repeated blocked actions	Review dependency, resourcing or ownership.	Action history and blocker duration.

Recommendation lifecycle: Evaluate rule → store/update recommendation with evidence snapshot → user accepts/dismisses/snoozes → accepted item becomes action → completion recorded → history retained. Deduplicate using rule type + linked entity + active condition. Never create duplicates on every refresh. Recommendations are decision-support prompts, not optimal, causal or validated predictions.

5. End-to-end workflows

Workflow A — Market opportunity to target account

1. Create a segment and record definition, geography, use case and evidence.
2. Enter market-sizing assumptions and sources; calculate estimates with visible labels.
3. Add competitors and compare capabilities/positioning.
4. Score segment and inspect component breakdown.
5. Create/import accounts and link them to the segment.
6. Review account fit score and missing research fields.
7. Select suitable accounts for opportunity creation.

Workflow B — Account to closed deal

1. Create/import account and record research evidence.
2. Calculate transparent fit score.
3. Create opportunity with value, stage, probability, expected close and next step.
4. Update stage/activity; retain stage history.
5. Recalculate pipeline and weighted pipeline.
6. Flag stale or overdue opportunities.
7. Closed Won converts to customer; Closed Lost records loss reason.

Workflow C — Closed Won to customer adoption

1. Link won opportunity to customer record.
2. Create onboarding plan from template.
3. Assign milestone owners and due dates.
4. Update status and blockers.
5. Enter/import adoption observations.
6. Recalculate health score with visible components and completeness.
7. Create risk recommendations and follow-up actions.

Workflow D — Business review to action

1. Create weekly/monthly review.
2. Display KPI snapshots and changes for period.
3. Record findings, decisions and unresolved issues.
4. Create actions with owner, priority and due date.
5. Monitor progress, overdue items and blockers.
6. Record completion notes and date.
7. Finalise review and preserve snapshot.

Workflow E — Signal to recommendation

1. Detect a documented business condition.
2. Store signal and evidence.
3. User reviews rationale and linked records.
4. User accepts, dismisses or snoozes.
5. Accepted recommendation becomes assigned action.
6. Completion/dismissal outcome remains available for review.

6. Data model and entity definitions

Use PostgreSQL with explicit relationships and foreign keys. UUID or integer primary keys are acceptable; choose one convention. Store timestamps in UTC and render using configured timezone.

Entity	Key fields and purpose
User	id, name, email, role, is_active, created_at.
MarketSegment	id, name, industry, geography, use_case, score, status.
MarketEstimate	segment_id, method, TAM/SAM/SOM, currency, assumptions_json, source_id, reviewed_at.
Competitor	name, website, category, notes; linked to segments.
SegmentCompetitor	segment_id, competitor_id, comparison_notes, position.
ResearchSource	title, URL, publisher, type, observed_at, confidence, notes.
Account	name, domain, industry, geography, segment_id, owner_id, fit_score, status.
Contact	account_id, name, title, department, optional email, notes.
Opportunity	account_id, owner_id, stage, value, currency, probability, close date, next step, activity date.
OpportunityStageHistory	opportunity_id, from_stage, to_stage, actor, changed_at.
Customer	account_id, opportunity_id, start date, plan, recurring value, renewal date, owner, lifecycle.
OnboardingPlan	customer_id, template, status, start date, target go-live.
OnboardingMilestone	plan_id, name, owner, due date, status, completion date, blocker.
AdoptionObservation	customer_id, period, active users, feature usage, frequency, source.
CustomerHealthSnapshot	customer_id, score, band, components_json, calculated_at.
CustomerRisk	customer_id, type, severity, description, owner, mitigation, status.
CostRecord	period, category, amount, currency, optional segment, notes.
BusinessTarget	metric_key, period, target, currency, optional segment/owner.
BusinessReview	title, period, owner, status, summary.
ReviewKpiSnapshot	review_id, metric_key, actual, target, variance, snapshot_at.
Decision	review_id, title, description, owner, date, status.
ActionItem	title, description, owner, priority, due date, status, linked entity, completion notes.
ProjectWorkstream	name, objective, owner, status, start/target date, progress.
ProjectMilestone	project_id, name, owner, due date, status.
Recommendation	rule_key, entity_type/id, title, rationale, evidence_json, severity, status.
AuditLog	actor, entity, action, before/after JSON, timestamp.

Data integrity rules

- Index frequently filtered fields: stage, owner, date, segment, customer status and action status.
- Validate required fields in API and database.
- Use decimal types for money, not floating point.
- Store currency codes with every monetary value.
- Never sum mixed currencies without explicit exchange rates and conversion date.
- Use constrained status values and soft-delete/archive where history matters.
- Store sources and assumptions for market estimates and competitor claims.
- Keep historical review and health snapshots.

7. Metrics, formulas and decision logic

Maintain a central metric registry. Each metric needs a key, display name, formula, unit, time basis, required inputs and missing-data behaviour.

Account fit score: Σ(criterion score × criterion weight) ÷ Σ(applicable weights), normalised to 0–100. Criteria may include segment, use-case, size, geography, urgency and data completeness. Show the breakdown and mark incomplete where required inputs are missing.

Customer health score: Configurable weighted combination of adoption, onboarding progress, support/risk signals, engagement and renewal proximity. Missing data must not silently count as healthy; show completeness and either adjust applicable weights or mark score incomplete.

Action priority score: Configurable impact × urgency × confidence model. Show inputs and label it as a prioritisation aid.

- Define time window and denominator for every percentage.
- Keep pipeline, bookings, recognised revenue and recurring revenue distinct.
- Return Not Available for empty denominators.
- Use canonical stage mapping for win rate and stage conversion.
- Do not infer recurring revenue from a one-time deal value.
- Use snapshots/event history for historical comparisons.
- Document defaults and allow admin configuration.

8. AI / recommendation layer and guardrails

Core calculations and rule triggers must be deterministic and testable; the app must work without an external LLM. Optional LLM features may summarise already-calculated evidence.

Permitted AI-assisted functions

- Summarise a market segment from stored research records and cite source records.
- Draft business-review narrative from KPI snapshots, decisions and notes.
- Explain why an account, opportunity or customer was flagged using rule evidence.
- Suggest possible next steps for human review.
- Convert meeting notes into draft action items requiring confirmation before saving.

Guardrails

- Never invent market statistics, competitor capabilities, pricing, customer feedback or metrics.
- Distinguish sourced facts, imported data, calculated metrics, assumptions and AI suggestions.
- Link generated summaries to records used.
- Never let AI overwrite structured source data automatically.
- Require human approval before actions or record changes.
- Do not send secrets or unnecessary personal data to an LLM.
- If evidence is absent, say information is unavailable.
- Keep deterministic rules separate from generated narrative.

9. Technical architecture and repository structure

Layer	Recommended technology	Responsibility
Frontend	React + TypeScript + Vite	Routing, layouts, forms, tables, filters, charts and interactions.
Styling	Tailwind CSS	Reusable responsive design system.
Backend	Python + FastAPI	REST APIs, validation, auth, permissions and business logic.
Database	PostgreSQL	Persistent relational data, constraints and transactions.
ORM / migrations	SQLAlchemy + Alembic	Models, migrations and query layer.
Analytics	SQL + Pandas + NumPy	Metric calculations, imports, transformations and data quality.
Charts	Plotly	Trend, comparison, funnel and segment charts.
ML (optional)	scikit-learn	Only when a defined use case and labelled data justify it.
Reporting	CSV/XLSX and PDF/print view	Exports and business-review outputs.
Hosting	Suitable cloud host; e.g. Render prototype	Deploy frontend/API/database with environment config.

Architecture principles

- Use a modular monolith initially; avoid unnecessary microservices.
- Keep business logic in backend services, not only frontend.
- Use predictable REST resources, validation and error responses.
- Keep secrets in environment variables; provide .env.example placeholders.
- Use database migrations and expose FastAPI OpenAPI/Swagger docs.
- Use background jobs only where imports, exports or scheduled refreshes need them.
- Provide health checks and structured logs.

Suggested repository structure

```
revopsiq/
frontend/src/
app/ # routes, layouts, providers
components/ # reusable components
features/
market-intelligence/
accounts/
pipeline/
customers/
business-performance/
business-operations/
recommendations/
services/ # API client
types/
backend/app/
main.py
api/ # routers and dependencies
core/ # config, auth, permissions
models/ # SQLAlchemy models
schemas/ # Pydantic schemas
services/ # business logic
rules/ # scoring and triggers
analytics/ # metric calculations
imports/ # CSV validation
reports/ # export generation
backend/migrations/
backend/tests/
data/seed/
docs/
product-requirements.md
data-dictionary.md
api-specification.md
metric-definitions.md
test-plan.md
.env.example
README.md
```

10. UI / UX requirements

Create a clean enterprise SaaS interface: information-dense but not cluttered, consistent, fast to scan and suitable for daily use. Avoid flashy gradients, excessive animation and decorative charts.

Primary navigation

- Overview
- Market Intelligence
- Accounts
- Sales Pipeline
- Customers & Adoption
- Business Performance
- Business Reviews
- Projects & Actions
- Recommendations
- Settings

Global interface requirements

- Persistent sidebar, clear page titles and global search for accounts, customers, opportunities and actions.
- Consistent filters, sorting, pagination, empty states and detail pages with activity/history and related records.
- Status labels must not rely on colour alone.
- Forms must show required fields, validation and success/failure feedback.
- Confirm destructive actions; use archive/undo where possible.
- Charts need labels, units, date range and metric definitions.
- Interactive cards must navigate or clearly appear non-interactive.
- Provide loading, empty, error and permission-denied states.
- Desktop-first with reasonable tablet responsiveness.

Overview dashboard

- KPI row: open pipeline, weighted pipeline, closed-won value, active customers, onboarding at risk, overdue actions.
- Market/pipeline: priority segments, pipeline by stage and segment.
- Customer: onboarding status, health distribution, renewals approaching.
- Operations: actions due, overdue actions, blocked projects.
- Decision panel: top active recommendations with evidence and navigation.

11. Demo data and import strategy

Use a synthetic dataset for a fictional B2B SaaS provider selling identity verification / fraud prevention technology to enterprise customers. This is a demonstration domain only; it must not imply affiliation with HyperVerge or access to its internal information.

- 8–12 fictional market segments across banking, fintech, marketplaces, telecom and insurance.
- 8–15 competitor profiles; label fictional attributes or attach public sources. Never invent real competitor pricing as fact.
- 40–60 fictional target accounts.
- 50–80 opportunities distributed across stages, values, probabilities and dates.
- 15–25 customers linked to won opportunities.
- Onboarding milestones in completed, in-progress, overdue and blocked states.
- At least six months of adoption observations for a subset of customers.
- Business targets and cost records for multiple periods and segments.
- Weekly reviews, decisions, projects and action items, including overdue and blocked examples.

- Seeded recommendations triggered by defined rules.

CSV import requirements

- Provide templates for accounts, opportunities, customers, adoption, targets and costs.
- Validate headers, types, required fields, duplicates, dates, currencies and foreign keys.
- Show preview and row-level errors before commit.
- Allow error report download.
- Report successful and failed rows; never hide partial imports.
- Record import batch ID, time, filename and row counts.

12. Security and reliability

- Hash passwords if authentication exists; never store plaintext.
- Enforce role-based authorization in backend endpoints.
- Validate/sanitise input; use ORM/parameterised queries.
- Keep secrets in environment variables.
- Use synthetic personal/customer data only in demo.
- Apply least privilege and avoid exposing sensitive fields in broad reports.
- Log record changes, imports, stage transitions, review finalisation and recommendation actions.
- Do not expose stack traces or secrets in user-facing errors.
- Document backup/restore assumptions; use HTTPS in deployment.

13. Testing and acceptance criteria

Backend tests

- Unit tests for account fit, customer health, weighted pipeline, win rate, stage conversion, CAC, LTV, gross margin, churn and variance.
- Test missing inputs, zero denominators, mixed currencies, invalid dates and negative values.
- API tests for create/read/update/archive across core entities.
- Permission tests for each role and protected endpoint.
- Workflow tests for Closed Won conversion and recommendation-to-action conversion.
- Import tests for valid files, malformed rows, duplicates and partial failures.
- Audit-log tests for important mutations.

Frontend / end-to-end tests

- Navigation and role access.
- Create/edit account and opportunity.
- Move opportunity through stages and verify pipeline totals.
- Convert won deal to customer and create onboarding milestones.
- Record adoption and verify health score explanation.
- Create review, decision and action.
- Accept recommendation and verify action creation.
- Filter/export report; verify empty, loading, validation and error states.

Release acceptance criteria

1. App starts locally from documented instructions.
2. Database schema can be created from empty database through migrations.
3. Seed script creates linked demo records.

4. All six modules have working screens connected to APIs and persistent records.
5. Core CRUD operations persist after refresh.
6. Scoring and financial metrics run in backend and are tested.
7. Opportunity, onboarding, review/action and recommendation workflows work end to end.
8. Search, filters, sorting and pagination work on relevant tables.
9. CSV import/export works with validation and error reporting.
10. Backend enforces role permissions.
11. README, .env.example, data dictionary, metric definitions and test instructions are complete.
12. No dead buttons, fake success messages, hardcoded KPI cards or unlabelled synthetic data remain.

14. Implementation phases

Phase	Scope	Exit criteria
0 — Design	Requirements, user flows, ERD, metric dictionary, API contracts and wireframes.	Approved build plan.
1 — Foundation	Repo, frontend shell, backend, PostgreSQL, migrations, roles, seed data and components.	App boots; database and seed data work.
2 — Accounts & pipeline	Segments, competitors, accounts, contacts, scoring, opportunities, stage history.	Account-to-opportunity flow tested.
3 — Customer operations	Customer conversion, onboarding, adoption, health and risk.	Won-deal-to-adoption flow tested.
4 — BI	Metric registry, targets, actuals, financial inputs, dashboards and exports.	KPIs trace to records; formulas tested.
5 — Business operations	Reviews, snapshots, decisions, projects, actions and blockers.	Review-to-action flow tested.
6 — Decision engine	Rules, evidence, deduplication, accept/dismiss and action conversion.	Signal-to-action flow tested.
7 — Hardening	Security, responsive polish, integration tests, docs, demo and deployment.	All release criteria met.

Time-limited strategy: Do not build six shallow modules simultaneously. Complete one vertical slice first: market segment → account → opportunity → customer → onboarding → KPI → recommendation → action. Then expand module breadth.

15. AI-agent operating instructions

Treat this synopsis as the single source of truth for the first implementation.

Before coding

1. Read the entire specification.
2. Inspect existing repository and report structure, stack, working features and broken areas. Do not overwrite working code blindly.
3. Create a checklist mapping every module and acceptance criterion to files, APIs, models and tests.
4. Propose database ERD and API resource list before implementation.
5. Identify ambiguities and state defaults explicitly; do not invent external facts.
6. Create phased plan and implement one complete vertical slice first.

During implementation

- Work in small phases; run tests and fix failures before proceeding.
- Do not create only frontend mockups. Functional screens must use real APIs and persistent data.
- Never hardcode dashboard metrics; calculate them from the database.
- No dead buttons. Implement the action or disable it with explanation.
- Keep business logic out of UI components.
- Use migrations and update data dictionary when models change.
- Use consistent naming, types, validation and error handling.

- Add automated tests for new logic and important endpoints.
- Label synthetic data visibly.
- Never claim deployment, integration, model accuracy or business impact that has not been verified.
- Do not add paid APIs, external integrations or credentials without approval.
- Do not make LLM a dependency for deterministic calculations.

Progress reporting

At the end of each phase report: completed features; files created/changed; database/API changes; tests run and results; known bugs/limitations; next phase. Distinguish implemented, tested, partial and planned functionality. If blocked, explain the blocker, propose the smallest safe alternative, preserve working code and document the limitation.

16. Deliverables, demo scenarios and definition of done

Required deliverables

- Working frontend and backend API.
- PostgreSQL schema and migrations.
- Seed data and reproducible seed script.
- CSV import templates and sample files.
- OpenAPI/API documentation.
- Product requirements and user-flow documentation.
- Database ERD and data dictionary.
- Metric/formula dictionary.
- Recommendation rule catalogue and scoring definitions.
- Test plan and automated tests.
- README with setup, environment, migration, seed, run, test and deployment steps.
- .env.example with placeholders only.
- Demo script and known-limitations document.

Demo 1 — Identify a market opportunity

1. Open a target segment; inspect definition, market-sizing assumptions and sources.
2. Compare competitors and inspect score breakdown.
3. Create or inspect target accounts linked to the segment.

Demo 2 — Prioritise and progress an account

1. Inspect account fit score and evidence.
2. Create opportunity with value, stage, probability and next step.
3. Move through stages and inspect history.
4. Show weighted pipeline and stale-deal alert.

Demo 3 — Onboard and monitor a customer

1. Mark opportunity Closed Won and create linked customer.
2. Generate onboarding plan and assign milestones.
3. Record adoption observations across periods.
4. Inspect health score, explanation and risk flags.

Demo 4 — Run a business review

1. Filter performance by period and segment.
2. Compare actuals with targets and inspect variance.
3. Create weekly review with KPI snapshots and findings.

4. Record decision and assign action.
5. Show action tracker and overdue/blocked work.

Demo 5 — Convert insight into execution

1. Open recommendation queue and inspect evidence.
2. Accept recommendation and convert to action.
3. Assign owner and due date.
4. Complete action and inspect history.

Final definition of done: A new user can start the app, load fictional demo data, research a segment, prioritise an account, manage an opportunity, convert a won deal into a customer, track onboarding and adoption, inspect performance, create a business review, receive an explainable recommendation, assign an action and verify completion — with persistent data, tested calculations and complete documentation.

Assumptions and boundaries

- RevOpsIQ is a portfolio project, not an official HyperVerge product or internal system.
- Identity verification / fraud prevention is a fictional demonstration domain; it does not imply access to HyperVerge internal information.
- All seed companies, customers, deals, usage observations and reviews are fictional unless explicitly sourced and labelled.
- Market estimates, health scores, fit scores, CAC, LTV and forecasts are estimates/configurable calculations; show assumptions and inputs.
- CRM, billing, support and product telemetry integrations are future work unless actually implemented and tested.
- LLM features are assistive and require human review; deterministic application functionality must work without them.

End of specification. Update this document and the tests when requirements change; do not rely on informal chat instructions as the only record of scope.